

P1725R0

Modular Topic Design

Published Proposal, 2019-06-17

This version:

<https://wg21.link/p1725>

Author:

[Christopher Di Bella](#)

Audience:

SG20

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

SG20 advised that [\[P1389R0\]](#) should move away from a linear stage-based model into a more modular topic-based model. This proposal aims to start the discussion for evaluating the the class of modules that are suitable for curriculum guidelines.

Table of Contents

- 1 Motivation
- 2 Design goals
- 3 A non-exhaustive list of modules
- 4 Bikeshedding
- 5 Acknowledgements

References

§ 1. Motivation

It was noted in the SG20 Kona face-to-face that learning is an iterative process, requiring malleability, and that the stages outlined in P1389 were too rigid and linear; this might be too restrictive on curriculum design as a result. This proposal sets out to ask SG20 to discuss the ideas behind a set of topic ‘[modules](#)’ where stages are embedded into a module. The benefit of the guidelines having sets of topics as opposed to sets of stages is that teachers will be able to formulate their own curricula more easily.

P1389R0 stated that that a student should be able to meet all of the objectives and satisfy all of the outcomes in Stage 1 before they are ready to progress to Stage 2, and so on. This means that there are some unrelated topics that are coupled together. In hindsight, the author of P1389 also acknowledges that the sets of stages is closer to a prescribed curriculum than intended. For example, some of P1389’s Stage 1 has algorithms, basic I/O, computation constructs, containers, contracts, and designing and using classes. None of these topics are strictly related, but Stage 2 is completely blocked until all the topics in Stage 1 are exhausted. Since P1389 advises derived types be introduced in Stage 2 -- and Stage 2 is blocked until Stage 1 is completed -- P1389 excludes the possibility of a curriculum introducing derived types as a part of the class design topic, despite it has no real dependency on basic I/O.

If P1389 or its successor were to advise modules of topics, where strict dependencies are outlined, then this would make it possible for both curriculum designs in the previous paragraph to co-exist and still conform to the recommendations outlined in the 'Standing Document for Teaching C++ to C++ Novices with Programming Experience' (P1389’s target). This will help shape the Standing Documents that SG20 outputs, so that they are significantly less prescriptive, and satisfy the advisory role outlined in the SG20 charter, as opposed to accidentally becoming a prescriptive curriculum.

§ 2. Design goals

It should be noted that while P1725 suggests a handful of modules and topics that fall within those modules, it is by no means an exhaustive list of either. Furthermore, P1725R0 does not make any attempt to draw a dependency graph among the topics listed in each topic. The author is requesting the help of SG20 to formulate an exhaustive list of modules and topics, as well as a strong dependency graph that will help teachers structure their curricula.

These modules are designed to have their own associated objectives and outcomes. A curriculum's objectives are essentially a contract between a student and the course. By the end of a course, a student should have developed certain knowledge, understanding, and skills about particular topics. These are assessable through course outcomes, which advise that a student should be able to do something related with that knowledge, or related to that skill. For example, Table 1 outlines a few objectives and outcomes from a hypothetical introductory C++ course.

Note: Table 1 is an example set of objectives and outcomes for a particular hypothetical course and is deliberately incomplete: P1725 is *not* proposing the contents of Table 1 be added to any document published by SG20.

Objective	Outcome
Students will develop	A student
1. knowledge and understanding about how software engineers build systems	P1.1 chooses the correct data structures, algorithms, libraries, and tools for engineering software P1.2 consults the most appropriate resources for building a system
2. knowledge and understanding about why testing is an important for software development	P2.1 designs unit-tests to validate that the individual components of their are correct P2.2 designs integration-tests to validate that their individual components work together to provide the correct solution
3. knowledge and understanding about why benchmarking is important for software development	P3.1 designs benchmarks to prove that one choice is "more efficient" than another
4. knowledge and understanding about software engineering tools	P4.1 chooses the correct tools to help solve the problem

5. skills in the C++ programming language

P5.1

writes programs that correctly solve the problem using C++

P5.2

writes programs that take advantage of modern C++ programming techniques

P5.3

reads and understands existing C++ code to understand the current state of a solution

P5.4

modifies or extends existing C++ code without the need for major rewrites to well-written code

§ 3. A non-exhaustive list of modules

Table 2 outlines a non-exhaustive list of modules and their related topics. The modules are not listed in a meaningful order; instead, they are listed so that the table is evenly formatted.

C++ compilation model	Error handling	I/O
• declarations • definitions • headers • modules • translation units	• contracts • exception handling • monadic programming	• character I/O • from-string conversions • file I/O • GUIs (using a third-party library) • to-string conversions
C++ object model	Functions	User-defined types
• copy semantics • constant expressions • constant objects • constructors • conversions • destructors	• calling functions • contracts • function templates • lambdas • member functions • overloading	• access • class templates • constructors • data members • derived types • destructors

- | | | |
|---|--|---|
| <ul style="list-style-type: none"> • free store • lambdas • move semantics • objects • pass-by-value • pass-by-reference • pointers • references • references-to-<code>const</code> • reference semantics • resources • rule-of-five • rule-of-zero • scope • smart pointers • special member functions • types • values • value semantics | <ul style="list-style-type: none"> • parameters • pass-by-value • pass-by-reference • returning multiple values • return value optimisation • user-defined literals • virtual functions | <ul style="list-style-type: none"> • friends • interface inheritance • member functions • <code>optional</code> and <code>variant</code> as sum types • rule-of-five • rule-of-zero • special member functions • static members • <code>struct</code>s and <code>class</code>es as product types • user-defined literals • variable templates • virtual functions |
|---|--|---|

Program design

- algorithms
- concepts
- containers
- contracts
- contributing to a pre-existing project
- error handling
- interface inheritance
- I/O design

Type system

- conversions
- fundamental types
- function overloading
- scoped enumerations
- template type parameters
- user-defined types

Tooling

- build system
- compiler
- continuous integration
- debugger
- formatter
- IDE
- libraries
- linter
- optimiser

• iterators • ranges • resource handling	• package manager • test tools • version control	
History of C++ No topic suggestions at present.	Interop with C No topic suggestions at present.	Undefined behaviour No topic suggestions at present.

An astute reader should notice that there are a few topics that are mentioned in multiple modules. This highlights that the above modules could perhaps be further refined *or* that certain topics cannot be taught in isolation. Given that features of C++ are designed to work with one another, the author believes that the latter case is more likely. The reader should also note that both ‘types’ and ‘user-defined types’ have been listed as modules *and* topics within other modules.

The author would like to propose that SG20 adopt the module/topic model -- even if it is in a different form to what is outlined above -- and begin discussing the kinds of modules desired, the topics underneath each module, the objectives for each module, and the relevant module outcomes associated with each objective.

§ 4. Bikeshedding

It’s been noted that the term ‘module’ is overloaded with core language. Terms such as ‘unit’ may be more applicable.

§ 5. Acknowledgements

The author would like to thank JC van Winkel for his direct feedback on this proposal, as well as the members of SG20 who provided feedback on P1389R0’s structure.

§ References

§ Informative References

[P1389R0]

Christopher Di Bella, Simon Brand, Michael Adams. [Standing Document for SG20: Guidelines for Teaching C++ to Beginners](https://wg21.link/p1389r0). URL: <https://wg21.link/p1389r0>